HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD

Prof. Yann Thoma

Formal verification
Verification of a locker
CERN training - November 2026

Context

We want to verify the behavior of a locker with the following interface :

```
entity locker is
port(
    clk_i     : in  std_logic;
    rst_i     : in  std_logic;
    keys_i    : in  std_logic_vector(9 downto 0);
    unlock_o : out std_logic
);
end locker;
```

The locker has a keypad with 10 keys, numbered from 0 to 9. The correct combination is `1-4-2-3`. The keys are represented by a 10-bit vector, with one bit per key. The locker opens when the correct combination is entered and remains open for one clock cycle. The keys may be held for any number of clock cycles, but there must not be a cycle with no key pressed between consecutive keys in the combination.
Develop a way to check the behavior of `unlock_o`. A sequence-based (regular-expression) approach may be useful. Consider how to verify that `unlock_o` remains low when the correct combination has not been entered.
The `ERRNO` generic parameter enables fault injection in the locker.
The following `ERRNO` values are used to test the assertions :

- `ERRNO` = 0 : correct behavior ;
- `ERRNO` in $[1, 7]$: faulty behavior.

The provided SBY script checks that verification passes for `ERRNO` = 0 and fails for values from 1 to 7. See the `locker.sby` file for the tasks that run verification with these values.

Version 2

In the `code2` folder, the assumption "there must not be a cycle with no key pressed between consecutive keys in the combination" is release. Can you find a way to check that ? Maybe some infinity will generate troubles.